

Simulador de TLB, Tabla de Páginas y Cache

Jesus Rachadell 31421494

1. El Problema

El proyecto consiste de desarrollar un sistema utilizando C++ para simular el funcionamiento de una Tabla de pagina, una TLB, y un Cache, manejando las políticas de remplazo típicas de este tipo de estructuras, además de medir la eficiencia de estos sistemas a través de conjuntos de datos. Los problemas a solucionar en este proyecto se presentan entonces como:

- Simular un entorno virtual con Tabla de páginas, TLB y caché
- Aplicar la técnica LRU para el remplazo de la información en TLB y caché
- Calcular la tasa de acierto de la TLB y caché
- Calcular el costo total en base a los ciclos que tomó al programa procesar la información
- Evaluar el impacto del fallo de página (PageFault) en el rendimiento del programa
- Permitir el cambio de estructuras Tabla de paginas, TLB y caché a partir del conjunto de Datos

El objetivo del programa es poder simular diferentes arquitecturas de Tabla de Pagina, TLB y Caché, y poder medir su eficiencia en base a distintos conjuntos de Datos. Permitiendo la evaluación y análisis de distintas arquitecturas para beneficio académicos.

2. Análisis de la solución

Lo primero a considerar es el flujo de información y como se va a manejar dentro del programa, para lograr eso se plantea el siguiente diagrama.

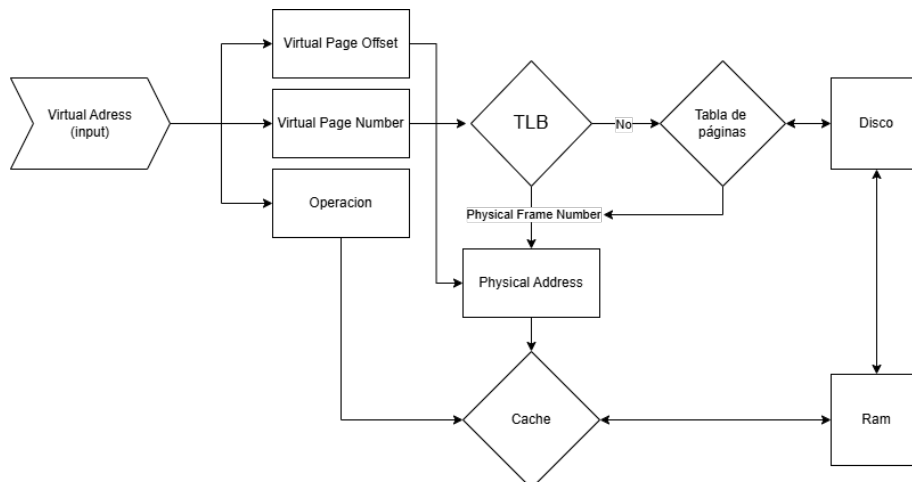


Figura 1: Flujo del programa

Como se muestra en el diagrama la información de entrada primero se divide, luego pasa por la TLB, en caso de fallo lo busca en la Tabla de Paginas, en caso de fallo trae lo necesario del Disco (PageFault) una vez conseguido el número del marco físico y con el offset se “crea” la dirección física la cuál es procesada por la cache.

Entonces la idea para la solución es crear un entorno virtual donde poder simular este proceso, donde a partir de un operador (read or write) y una dirección en hexadecimal se pueda simular como actuaría un entorno real, con el agregado de poder modificar la arquitectura usada del entorno desde el mismo documento de entrada.

3. Estructuras de Datos

Estructura de la TLB

```
1 struct TLBEntry{
2     uint32_t vpn;
3     uint32_t pfn;
4     bool valid = false;
5     int activationTime;
6 };
7
8 class TLB{
9 private:
10     std::vector<TLBEntry> entries;
11     uint32_t nEntries;
12     uint32_t hits = 0;
13     uint32_t misses = 0;
14     int currentTime = 0;
15
16 public:
17     // Funcion Constructura
18     TLB(uint32_t size);
19
20     // Revisa si existe una traduccion guardada
21     uint32_t lookup(uint32_t vpn);
22
23     // En caso de TLBmiss, se busca en tabla de paginas y se intercambia
24     // con esta funcion
25     void insert(uint32_t vpn, uint32_t pfn);
26
27     // Tasa de aciertos TLB
28     double hitRate() const;
29 };
```

Estructura de la Tabla de Páginas

```
1 struct PageEntry{
2     uint32_t pfn;
3     bool inRam = false;
4     bool valid = false;
5 };
6
7 class PageTable{
8 private:
9     std::vector<PageEntry> table;
10    int pageFaults = 0; // variable de control
11    bool laFault = false; //LastAccessWasFault
12
13 public:
14     PageTable(uint32_t size); // Funcion Constructura
15
16     uint32_t getPfn(uint32_t vpn); //Busca pfn
17
18     void handleFault(uint32_t vpn); //Maneja el error
19
20     int getPageFault() const; // funcion get de pageFaults
21
22     bool getlaFault(); // funcion get de LastAccessWasFault
23 };
```

Estructura de la Cache

```
1 struct CacheEntry{
2     uint32_t tag = 0;
3     int activationTime = 0;
4     bool dirtybit = false;
5     bool valid = false;
6 };
7
8 struct MissResult{
9     int penalty;
10    CacheEntry& line;
11 };
12
13 class Cache{
14 private:
15     std::vector<std::vector<CacheEntry>> sets;
16
17     uint32_t numSets;
18     uint32_t associativity;
19     uint32_t blockSize;
20
21     uint32_t offsetBits;
22     uint32_t indexBits;
23
24     int hits = 0;
25     int misses = 0;
26     int currentTime = 0;
27 public:
28     // Funcion Constructura
29     Cache(uint32_t cacheSize, uint32_t assoc, uint32_t bSize);
30     // Hace la operacion
31     int access(uint32_t pfn, char operation);
32     // Maneja el error
33     MissResult handleMiss(uint32_t index, uint32_t tag);
34     // Tasa de Aciertos Cache
35     double getHitRate();
36 };
```

4. Algoritmos claves

Algorithm 1 Buscar_PFN_En_TLB()

```
1: Entrada: VPN (Virtual Page Number)
2: for all entry  $\in$  entries do
3:   if (entry.valid  $\wedge$  entry.vpn = VPN) then
4:     entry.activationTime  $\leftarrow$  currentTime
5:     hits ++
6:     return entry.pfn {Physical Frame Number}
7:   end if
8: end for
9: misses ++
10: return -1
```

Algorithm 2 Insertar_en_TLB

```
1: Entrada: VPN, PFN
2: lruIndex  $\leftarrow$  0
3: oldestTime  $\leftarrow$  entries[0].activationTime
4: for i = 0; i < nEntries; i ++ do
5:   if ( $\neg$ (entries[i].valid)) then
6:     entries[i].vpn  $\leftarrow$  VPN
7:     entries[i].pfn  $\leftarrow$  PFN
8:     entries[i].valid  $\leftarrow$  true
9:     entries[i].activationTime  $\leftarrow$  currentTime
10:    return
11:  end if
12:  if (entries[i].activationTime < oldestTime) then
13:    oldestTime  $\leftarrow$  entries[i].activationTime
14:    lruIndex  $\leftarrow$  i
15:  end if
16: end for
17: entries[lruIndex].vpn  $\leftarrow$  VPN
18: entries[lruIndex].pfn  $\leftarrow$  PFN
19: entries[lruIndex].activationTime  $\leftarrow$  currentTime
```

Algorithm 3 Conseguir_PFN_en_Tabla_de_Paginas

```
1: Entrada: VPN
2: if VPN  $\geq$  table.size()  $\vee$   $\neg$ (table[VPN].valid) then
3:   lastAccessWasFault  $\leftarrow$  false
4:   return -1 {SegmentFault}
5: end if
6: if ( $\neg$ table[VPN].inRam) then
7:   pageFault ++
8:   handleFault(VPN)
9:   lastAccessWasFault  $\leftarrow$  true
10:  return table[VPN].pfn
11: end if
12: lastAccessWasFault  $\leftarrow$  false
13: return table[VPN].pfn
```

Algorithm 4 Acceso_Cache

```
1: Entrada: PFN, operador
2: currentTime ++
3: adressWithoutOffset  $\leftarrow$  pfn  $\gg$  offsetBits
4: index  $\leftarrow$  adressWithoutOffset & (numSet - 1)
5: tag  $\leftarrow$  adressWithoutOffset  $\gg$  indexBits
6: for all line  $\in$  sets[index] do
7:   if (line.valid && line.tag = tag) then
8:     line.activationTime = currentTime
9:     hits ++
10:    if (operador = 'W') then
11:      line.dirtybit  $\leftarrow$  true
12:    end if
13:    return 1
14:  end if
15: end for
16: misses ++
17: result = handleMiss(index, tag)
18: if (operador = 'W') then
19:   line.dirtybit  $\leftarrow$  true
20: end if
21: return result.penalty
```

5. Aclaraciones

- **Uso del uint32_t:** Permite trabajar siempre con 32 bits sin importar la arquitectura (SO). el uso de la etiqueta unsigned permite el desplazamiento de bits a conveniencia lo cual es perfecto para la estructura de este trabajo
- **Hit rate:** Tanto para el hit rate de la TLB y el hit rate de la cache se usa la siguiente formula $hits/(hits + misses)$
- **Fallos de Página:** Como no poseemos RAM ni Disco, cuando hay un fallo de página asumimos que la encontró y usamos lo inicializado en pfn. Esto se hace para evitar complicaciones con los fallos, y se ve reflejado en la estadística “Ciclos totales” donde cada PageFault le toma 1.000.000 de ciclos por fallo.
- **Herramientas:** Como podrá observar dentro de la carpeta del proyecto existe una carpeta llamada “Generador” y dentro un archivo .cpp, es una herramienta externa que use para generar el formato de los Datos (que será cubierto más adelante). No es parte del proyecto como tal pero la dejé ahí en caso de que quiera generar sus propios datos, solo espero que entienda que los valores ingresados (exceptuando el número de instrucciones) es obligatorio usar potencias de dos, hacerlo de manera diferente podría ocasionar fallos inesperados.
- **Fallos de Segmentos:** Por como está realizado el generador de Datos y como se interpretan en el simulador es matemáticamente imposible llegar a este caso. Pero de todas maneras se incluyó un manejador de fallos de segmento en caso que decida experimentar más allá del límite.
- **LRU:** Para lograr que el LRU funcione se utilizó un sistema de tiempo, cada que hay un acceso se genera aumenta un contador, ese valor se le asigna a la línea accedida. Cuando se necesita hacer un cambio entonces el programa revisa quien tiene el valor más bajo y lo cambia respectivamente. Se puede observar en las variable *activationTime* y *currentTime* que se encuentran en la TLB y en la Cache.

6. Pruebas Realizadas

6.1. Datos

Antes de pasar a las pruebas quisiera explicar brevemente como funciona la creación de los datos, lo primero que verá en cualquier archivo de Datos es que hay una cabecera que dicta la arquitectura:

- **Tamaño de la TLB.**
- **Tamaño de la página.**
- **Tamaño de la Cache.**
- **Nivel de asociatividad.**
- **Tamaño del bloque.**

Luego de eso el archivo se compone de n instrucciones (1000 en los primeros 4 casos y 1000000 en el último) con un operador para indicar su papel en la cache, opte por este modelo ya que junto a la herramienta del generador se pueden probar distintas arquitecturas sin tener que cambiar la arquitectura dentro del código, haciéndolo mucho más cómodo de manejar.

Finalmente en la realización de estos archivos de Datos se incluyó la semilla “123” para la conformidad de los Datos, así poder evaluar cada arquitectura de manera justa. Dentro de la carpeta del proyecto se encuentran los datos con los cuales se evaluaron las arquitecturas, se indica primero el nombre “Dato” luego la letra correspondiente a la arquitectura y finalmente el tipo de dato por el número correspondiente.

	Tipo 1	Tipo 2	Tipo 3	Tipo 4	Tipo 5
Número de instrucciones	1000	1000	1000	1000	1000000
Aleatoriedad (0.0-1.0)	0.0	0.1	0.4	1.0	0.15

Cuadro 1: Datos y características

Como se puede observar la diferencia entre los tipos de datos es su aleatoriedad y en el último su tamaño, entre más aleatoriedad mayor serán las distancias que las instrucciones tienen las unas con las otras, es decir menos localidad espacial y/o temporal, gracias a eso podemos evaluar la efectividad de las distintas arquitecturas.

6.2. Arquitecturas

Se presenta una tabla con las características de las 5 arquitecturas en las que se estarán realizando las pruebas, cada una identificada con una letra, la elección de estas arquitecturas se pensó para estresar el programa de distintas maneras, siendo A la versión base, B la versión de mayor capacidad, C la versión menos capacitada, D la versión para ver que pasaría si la asociatividad disminuye pero aumenta el tamaño del bloque, y finalmente E la versión que busca estresar a la TLB.

	Arquitectura A	Arquitectura B	Arquitectura C	Arquitectura D	Arquitectura E
Tamaño de la TLB	16	64	4	16	2
Tamaño de la página(bytes)	4096	8192	2048	4096	4096
Tamaño de la cache(bytes)	16368	65536	2048	16368	131072
Nivel de asociatividad	4	8	1	2	8
Tamaño del bloque(bytes)	64	128	16	256	64

Cuadro 2: Arquitecturas

6.3. Resultados

Se presentan 3 tablas representado cada una el hit rate de la TLB, la hit rate de la Cache, y la cantidad de ciclos totales por archivo de dato en cada arquitectura respectivamente.

%	Arquitectura A	Arquitectura B	Arquitectura C	Arquitectura D	Arquitectura E
Datos 1	99.2 %	99.6 %	98.5 %	99.2 %	99.2 %
Datos 2	89.7 %	90.2 %	88.8 %	89.7 %	89.7 %
Datos 3	58.6 %	58.7 %	58.3 %	58.6 %	58.6 %
Datos 4	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %
Datos 5	84.5 %	84.8 %	83.9 %	84.5 %	84.5 %

Cuadro 3: Hit Rate de la TLB

%	Arquitectura A	Arquitectura B	Arquitectura C	Arquitectura D	Arquitectura E
Datos 1	76.4 %	76.4 %	15.7 %	94.1 %	52.8 %
Datos 2	69.5 %	69.5 %	14.5 %	85.7 %	48.1 %
Datos 3	45.1 %	45.1 %	8.9 %	56.2 %	32.1 %
Datos 4	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %
Datos 5	65.2 %	65.2 %	13.2 %	80.1 %	45.2 %

Cuadro 4: Hit Rate de la Cache

	Arquitectura A	Arquitectura B	Arquitectura C	Arquitectura D	Arquitectura E
Datos 1	8.031.956	4.025.360	15.102.942	8.010.533	8.048.720
Datos 2	103.040.992	98.032.197	112.104.133	103.024.054	103.053.278
Datos 3	414.070.037	413.059.138	417.110.172	414.061.348	414.068.807
Datos 4	1.000.119.400	1.000.110.300	1.000.119.100	1.000.121.200	1.000.100.000
Datos 5	2.551.875.235	3.113.681.967	772.841.863	2.532.967.230	2.573.725.701

Cuadro 5: Ciclos totales por archivo

7. Análisis y Observaciones

7.1. Observaciones

Como puede observar el número de ciclos por archivo se dispara enormemente esto es debido a que al iniciar una pagina que no estaba dispuesta en la tabla se suma 1.000.000 de ciclos (ir a Disco y volver) y por la arquitecturas con la que se trabaja es bastante común que a mayor aleatoriedad más fallos de páginas se generen y dispare la cantidad de ciclos, una forma de mejorar esto es aumentando el tamaño de la página, esto ocasionará que sea lento al principio pero una vez cargadas las páginas la reducción en ciclos será notorias. Otra solución aunque pueda ser un problema si no tiene en consideración del programador es cargar varias paginas vecinas para aprovechar la localidad espacial, esto para sistemas planificados puede reducir por mucho la cantidad de fallos, sin embargo en sistemas donde las instrucciones sea aleatorias y no tengan un sentido como esta puede generar mucha lentitud, por eso se optó por dejar fuera está opción sin embargo nos pareció correcto mencionar que se tuvo en cuenta durante el desarrollo.

Con el archivo Datos 4 puede observar que tanto la hit rate de la TLB y la hit rate de la Cache, caen a cero, esto debido a que el archivo con la aleatoriedad al máximo hace casi imposible que se carguen direcciones semejantes arruinando completamente la localidad espacial y temporal. Sin embargo ese nivel de aleatoriedad es casi nulo en sistemas reales ya que el Sistema Operativo suele usar las direcciones de tal manera de poder sacarle el máximo provecho a la máquina, el ejemplo de Datos 4 es solo un caso al extremo.

En el caso de los Datos 5 se puede observar una discrepancia en los ciclos totales de la arquitectura B y C, pese a la lógica que venia arrastrando en los demás Datos, en este caso la arquitectura C tiene menos ciclos totales que las demás arquitecturas y en cambio la arquitectura B se dispara, haciendo varias pruebas con otros Datos he concluido que por como están contruidos los Datos 5, es posible que el orden de estos Datos haya generado que sea más eficiente en la arquitectura C que en la D, probando otros tipo de Datos el resultado suele ir con la lógica que D es menos costoso que C.

7.2. Análisis

El código cumple con lo presentado como solución permitiendo poder simular un entorno virtual con Tabla de Páginas, TLB y caché los cuales poseen una política de remplazo LRU que permite aprovechar la localidad temporal, a su vez este simulador permite recibir entrada desde archivos donde a partir de una dirección virtual poder acceder al entorno y ser procesada correctamente. Se implemento un sistema de hits y misses que permite conocer la hit rate de la TLB y Cache, el cual nos presenta la eficacia de estos sistemas. Se implemento un sistema realista de ciclos el cual aumenta dependiendo de la instrucción y su situación dentro del entorno permitiendo observar que pasaría en un sistema real y permitiendo evaluar el impacto real de un fallo de página en el rendimiento. El simulador permite cambiar de forma sencilla la arquitectura que se desea evaluar logrando el objetivo de ser flexible y re utilizable.

En los resultados se puede observar como dependiendo de que arquitectura se use y el enfoque puede generar resultados variados, si queremos evaluar que pasaría en un sistema realista lo mejor sería fijarse en los resultados de las arquitecturas cuando se les pasa los Datos 1, 2 y 5, ya que los Datos 3 y 4 sirven para ilustrar lo que pasaría en un sistema estresado por aleatoriedad. En un sistema real los programas están contruidos de tal manera que aprovechan la localidad temporal y espacial asemejándose a los Datos 1, 2, y 5, bajo esta premisa se puede observar que Arquitecta D posee una diferencia notable en cuanto a el hit rate de la cache, esto es debido a la gran diferencia en el tamaño de bloque, sin embargo pese al alto acierto en ambas TLB y Cache la cantidad de ciclos se dispara haciendo su uso bastante costoso de todas formas. Si vemos la arquitectura B posee hit rates similares a la A, y suele mantenerse estable (sin contar los Datos 4) sin embargo donde brilla es en el campo de los ciclos totales, aquí se puede observar una reducción importante de los ciclos gracias a su capacidad de página que se discutió antes. Para finalizar la arquitectura E presenta una premisa interesante, que pasaría si reducimos la TLB pero aumentamos en cantidad el tamaño de la cache, pesé a su gran capacidad de cache la arquitectura presento muchos problemas incluso en situaciones secuenciales (Datos 1) se entiende entonces que para poder aumentar la eficacia de una arquitectura es importante aumentar a la vez de la cache aumentar el tamaño de bloque.

Podemos concluir que el simulador ha demostrado poder simular correctamente el comportamiento de la Tabla de Páginas, la TLB y la cache, permitiendo su utilidad como herramienta para probar distintas arquitecturas y ver que tan eficaces son en distintos casos.